

Lists in Python

part two

7. List methods

List methods

dot notation

The list methods are applied on the list instance, using the *dot notation*. As follows:

```
1 cities = ["Stockholm", "Oslo", "London"]
2 cities.reverse()
3 print(cities)
```

```
['London', 'Oslo', 'Stockholm']
```

Contrast this with how the **built-in functions** are called:

```
1 cities = ["Stockholm", "Oslo", "London"]
2 nr_of_cities = len(cities)
3 print(nr_of_cities)
```

List methods

Note the difference between string methods and many list methods:

Strings are **immutable** (=cannot be changed), and many string methods return **a copy** of the original string.

```
1 original_hello = "Hello World!"
2 upper_hello = original_hello.upper()
3 print("upper_hello: " + upper_hello)
4 print("original_hello: " + original_hello)
```

upper_hello: HELLO WORLD!

original_hello: Hello World!

Lists are **mutable** (=can be changed), and most list methods **modify** the list (instead of returning a copy).

```
1 cities = ["Stockholm", "Oslo", "London"]
2 cities.reverse()
3 print(cities)
```

['London', 'Oslo', 'Stockholm']

List methods

Strings are **immutable** (=cannot be changed), and many string methods return a **copy** of the original string.

```
1 original_hello = "Hello World!"
2 upper_hello = original_hello.upper()
3 print("upper_hello: " + upper_hello)
4 print("original_hello: " + original_hello)
```

upper_hello: HELLO WORLD!

original_hello: Hello World!

Many (but not all) list methods that **modify** the list return None.

```
1 cities = ["Stockholm", "Oslo", "London"]
2 return_value = cities.reverse()
3 print(cities)
4 print(return_value)
```

['London', 'Oslo', 'Stockholm']

None

Python documentation

<https://docs.python.org/3/library/stdtypes.html#lists>

Mutable Sequence Methods

Mutable sequence types also support the following methods:

`sequence.append(value, /)`

Append *value* to the end of the sequence. This is equivalent to writing `seq[len(seq):len(seq)] = [value]`.

`sequence.clear()`

Added in version 3.3.

Remove all items from *sequence*. This is equivalent to writing `del sequence[:]`.

`sequence.copy()`

Added in version 3.3.

Create a shallow copy of *sequence*. This is equivalent to writing `sequence[:]`.

List methods

When going through some of the string methods, we will use the following division of the methods:

- ▶ Methods that do not modify the list
- ▶ Methods that modify the list

8. List methods that do not modify the list

List methods that do not modify the list

count()

count(value) returns the number of occurrences of a value in a list.

```
1 cities = ["Stockholm", "Oslo", "London", "Stockholm"]
2 print(cities.count("Stockholm")) no of times it came in list
3 print(cities.count("London"))
4 print(cities.count("Helsinki"))
5 print(cities)
```

2

1

0

['Stockholm', 'Oslo', 'London', 'Stockholm']

List methods that do not modify the list

index()

index(value) returns the index of the first occurrence of value in sequence.

```
1 cities = ["Stockholm", "Oslo", "London", "Stockholm"]
2 print(cities.index("Stockholm"))
3 print(cities.index("London"))
4 print(cities)
5 print(cities.index("Helsinki"))
```

0

2

['Stockholm', 'Oslo', 'London', 'Stockholm']

ValueError: 'Helsinki' is not in list

List methods that do not modify the list

`copy()` returns a copy of the list.

```
1 cities = ["Stockholm", "Oslo", "London", "Stockholm"]
2 copy_of_cities = cities.copy()
3 copy_of_cities[0] = "STOCKHOLM"
4 print(cities)
5 print(copy_of_cities)
```

```
['Stockholm', 'Oslo', 'London', 'Stockholm']
['STOCKHOLM', 'Oslo', 'London', 'Stockholm']
```

Using `[:]` is equivalent to using the method `copy()`:

```
1 cities = ["Stockholm", "Oslo", "London", "Stockholm"]
2 copy_of_cities = cities[:]
3 copy_of_cities[0] = "STOCKHOLM"
4 print(cities)
5 print(copy_of_cities)
```

```
['Stockholm', 'Oslo', 'London', 'Stockholm']
['STOCKHOLM', 'Oslo', 'London', 'Stockholm']
```

9. List methods that modify the list

List methods that modify the list

`reverse()` reverses the list (and returns `None`).

```
1 cities = ["Stockholm", "Oslo", "London"]
2 return_value = cities.reverse()
3 print(cities)
4 print(return_value)
```

```
['London', 'Oslo', 'Stockholm']
```

```
None
```

So there is no `point of caring about the return value`. So the code would look like the following:

```
1 cities = ["Stockholm", "Oslo", "London"]
2 cities.reverse()
3 print(cities)
```

```
['London', 'Oslo', 'Stockholm']
```

List methods that modify the list

append()

append(value) appends value to the end of the list.

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 print(cities)
3 cities.append("Berlin")
4 print(cities)
```

['Stockholm', 'Oslo', 'Stockholm']

['Stockholm', 'Oslo', 'Stockholm', 'Berlin']

Both take $O(n)$ time for n items, so they have the same Big-O computational complexity. However, `extend()` is usually faster in practice because it is implemented efficiently at the Python level/C level and avoids the overhead of repeatedly executing the loop in Python.

List methods that modify the list

append() and extend()

extend() is generally more efficient and simpler than using a nested loop with append() when you want to add all elements from one iterable.

If you append a list to a list, you will get a nested list:

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 cities.append(["Berlin", "Warsaw"])
3 print(cities)
```

```
['Stockholm', 'Oslo', 'Stockholm', ['Berlin', 'Warsaw']]
```

use append if you want to add 1 item, and extend if want to add more than one item, then use extend

If you don't want a nested list, you can use extend instead.

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 cities.extend(["Berlin", "Warsaw"])
3 print(cities)
```

```
['Stockholm', 'Oslo', 'Stockholm', 'Berlin', 'Warsaw']
```

List methods that modify the list

pop()

When using `pop()` it's meaningful to retained the returned value, because it contains the element that has been removed.

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 removed_city = cities.pop(1)
3 print(removed_city)
4 print(cities)
```

Oslo

['Stockholm', 'Stockholm']

List methods that modify the list

When using `pop()` it's meaningful to retained the returned value, because it contains the element that has been removed.

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 removed_city = cities.pop(1)
3 print(removed_city)
4 print(cities)
```

Oslo

['Stockholm', 'Stockholm']

10. Multiple reference to the same list

Multiple references to the same list

`cities` and `cities_2` refer to the same list.

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 cities_2 = cities
3 cities.append("Berlin")
4 print(cities)
5 print(cities_2)
```

```
['Stockholm', 'Oslo', 'Stockholm', 'Berlin']
['Stockholm', 'Oslo', 'Stockholm', 'Berlin']
```

Make a copy, if you want two separate lists:

```
1 cities = ["Stockholm", "Oslo", "Stockholm"]
2 cities_2 = cities.copy() shallow copy
3 cities.append("Berlin")
4 print(cities)
5 print(cities_2)
```

```
['Stockholm', 'Oslo', 'Stockholm', 'Berlin']
['Stockholm', 'Oslo', 'Stockholm']
```

11. sort() and sorted()

sort() and sorted()

sorted() is a built-in function and sort() is a string method.

```
1 cities = ["Stockholm", "Oslo", "London"]
2 cities.sort()
3 print(cities)
```

```
['London', 'Oslo', 'Stockholm']
```

```
1 cities = ["Stockholm", "Oslo", "London"]
2 sorted_cities = sorted(cities)
3 print(cities)
4 print(sorted_cities)
```

sort() changes the original list, while sorted() creates a new sorted list and keeps the original unchanged.

```
['Stockholm', 'Oslo', 'London']
```

```
['London', 'Oslo', 'Stockholm']
```

sorted() is more general. It can sort other iterables such as tuples and strings too, while .sort() is specifically a list method.